OPERATING SYSTEMS · FALL 2026 · PROJECT 2

Variable-Size Cache Manager

Manage a fixed memory pool for cached objects and decide what to remove when a new object needs room.

Why this problem matters

A service reads an image from storage and keeps a copy in memory. When someone requests that image again, the service can use the copy instead of reading from storage a second time. The service cannot devote all its memory to cached images, however; it needs memory for its other work. Its cache has a limited budget.

Now the service encounters another image it would like to keep. Several images are already cached, and the new one will not fit alongside them. Removing an older image may make room, but that image may be the next one requested. In that case the service must read it from storage again. The cache has to make a decision now without knowing what people will request later.

The sizes of the images make this more than a choice of what to discard. Free space may be scattered between cached images. The cache might have enough unused bytes in total but no single region large enough for the new image. Removing an image beside a free region can create one large usable block; removing an image elsewhere might not. The service must decide both which object to remove and where to put the new one. An earlier choice of location can determine whether a later object fits without any eviction at all.

In `cachelab`, you will build a small, controlled version of this cache manager. Its objects and requests are simulated: the program does not read images or measure storage delays. Instead, a fixed pool and a repeatable trace expose the decisions that a real cache must make under a memory limit. This is why the problem belongs in a memory-management module. Free-space management determines which regions can hold a new object; replacement policy determines which existing objects surrender their regions.

The program model

The program uses a pool of a specified number of bytes. This pool is allocated once and never grows. It processes a *trace*, a file containing one cache request per line. A trace is a repeatable workload: with one client, you can give the same requests to several policy combinations and attribute differences in the results to their decisions rather than to different inputs.

```
PUT logo.png 100
GET logo.png
```

```
PUT banner.jpg 200
GET missing.png
```

`PUT key size` asks the cache to reserve space for an object with that key and requested size. A later `GET key` reports a **hit** if the object is still present and a **miss** if it is not. A GET does not change the pool: it neither allocates an object on a miss nor makes room for one. A hit does, however, record that the object was referenced, information the clock eviction policy can use.

In this model, a PUT specifies an object's size without providing its data, and a GET checks for its presence without fetching it on a miss. The trace supplies the operations needed to study space and replacement decisions. The cost of reading an image again explains why a miss matters in a real service; this program reports the miss without measuring that cost.

The finished cache has three cooperating parts. A pool allocator keeps track of regions of memory. Cache records connect keys to the objects occupying those regions and keep the information needed to choose a victim. Request handling joins the two: it looks up keys, places new objects, and evicts existing ones when allocation fails. The central student work is to make the embedded list handle allocation and freeing correctly, then make placement and eviction work together across requests.

The embedded block list

From an ordinary linked list to a pool

In an ordinary C linked list, each node can have its own `malloc` allocation. Nodes may be at unrelated addresses; each contains a pointer to the next node. This allocator uses the same idea of linked nodes but places the nodes *inside the memory it manages*. The program makes one allocation for the pool. Every list node, called a block header, occupies part of that pool, immediately before the region of bytes it describes. Storing another object does not call `malloc` for another pool block; it carves a block from space already in the pool.

```
Ordinary list:   [ node ] ----→ [ node ] ----→ [ node ]
                   separate malloc allocations, possibly far apart

Pool list:       [ header | payload ][ header | payload ][ header | payload ]
                 └──────────────── one pool allocation ────────────────┘
                headers link to the next header in pool address order
```

The links still form a list. What changes is where its nodes live and how a new node is created.

Each header records how much payload capacity follows it, whether that block is free, and the location of the next header. A header consumes pool space too. At startup the list contains one header followed by one free region spanning the remainder of the pool. The list contains **every block, free or allocated**; it is not a separate list of only the holes. Following its links walks across the pool from low addresses to high ones.

How the list changes

Consider the same pool at four moments. In the sketches, `H` is a header, and the boxes following it are the space described by that header. The widths are illustrative, not byte-for-byte measurements:

```
Initially:
  [ H: free ][                 free space                 ]

After storing A, then B:
  [ H: A ][ A space ][ H: B ][ B space ][ H: free ][ free ]

After B is freed and merges with the free block after it:
  [ H: A ][ A space ][ H: free ][  B space + free space  ]

After A is freed and merges with that block:
  [ H: free ][                whole pool                 ]

Header links after the two allocations:  H(A) → H(B) → H(free)
After freeing B:                         H(A) → H(free)
After freeing A:                         H(free)
```

Splitting creates headers inside the pool. Coalescing removes a boundary between adjacent free blocks; no bytes move.

The first allocation uses part of the initial free block. If enough space remains for another header and a usable payload, the allocator puts a new header at the start of that remainder. A second allocation can split the remainder again. The headers for A and B describe allocated blocks; the last header describes the still-free block. The list links all three in pool address order. If the remainder is too small for a header and a usable payload, there is no new block: the allocated block keeps the extra capacity.

Now suppose B is freed. Its header changes to free. Since the block immediately after B is already free, the two regions can become one: B's header describes their combined space, and its link skips the following header. If A is later freed, its block can merge with this larger free region in the same way. A header removed from the list no longer marks a block boundary; its bytes become part of the merged region. Coalescing changes the description of the pool. It does not copy the objects elsewhere.

The list's links point forward only. If a freed block has a free block *before* it, the allocator must find that predecessor by walking from the start of the list. When a freed block has free neighbors on both sides, it can merge forward first and then let its predecessor absorb the result. Understanding these changes to header sizes and links is essential: an incorrect list can lose usable bytes or describe overlapping blocks even if the total requested payload count looks plausible.

Maintaining this list is part of the work of the project. Searching for a fitting block, creating a header when a block is split, and removing a header when blocks merge are changes to the pool's actual representation, not bookkeeping added

after an allocation. The placement policies need a correct list to search, and eviction relies on freeing blocks into that same list.

A block's size describes its *payload capacity*, not its header. An object has a requested size, which can be smaller than the capacity reserved for it when alignment or a too-small remainder prevents an exact split. These distinctions explain why a pool of 1,024 bytes cannot necessarily hold 1,024 bytes of object payloads. They also explain why reporting the size of a cached object is different from reporting the capacity of its block. The pool tutorial develops the pointer arithmetic and alignment rule; here the concern is what each operation does to the pool's layout.

Allocation and fragmentation

An allocation must find *one free block* whose payload capacity can accommodate the request. It cannot combine two distant free blocks by adding their sizes. For example, imagine a free region with 80 bytes of payload capacity and another with 60 bytes, separated by an allocated object B. Together they have 140 bytes, but neither can hold a new 100-byte object.

```
[ A ][ free: 80 ][ B ][ free: 60 ][ C ]
                  PUT D needs one block of 100
```

Sizes are illustrative free payload capacities. Headers and alignment also occupy space in the real pool.

If B leaves, its block can join the free blocks on either side, making a large contiguous region. If C leaves instead, the pool has a different arrangement of free space. The allocator's answer to “does D fit?” depends on the largest usable region, not just the sum of all free bytes. This is *external fragmentation*: useful capacity exists, but is divided among separate regions.

Splitting and coalescing change this arrangement over time. Placement affects where the next gaps will be; freeing a block may join a gap to one or both of its neighbors. The embedded list makes those changes visible. Because it contains allocated blocks as well as free ones in address order, the allocator can tell which blocks are adjacent in the pool and which free regions can actually merge.

There is also a smaller-scale cost inside the blocks. Every new block needs a header, and split points must keep later headers properly aligned. If the remainder of a free block is too small to form a valid new block, the allocation keeps that extra capacity. Those bytes are unavailable to other objects even though the requested object does not use them. The requested sizes printed in cache events therefore need not add up to every occupied byte in the pool.

Cache entries and key lookup

The embedded block list answers questions about *space*: which blocks are free, how large they are, and which ones are neighbors. It cannot answer a different question: “Is `logo.png` still cached?” Block headers do not contain keys. A cache needs a record for each stored object that connects its key to its allocated pool region.

That record is a cache entry. It contains the key, the object's requested size, a pointer to its pool allocation, and information used for eviction. The entries and their lookup structures live outside the fixed pool; the pool is charged for block headers and reserved payload space, not for key strings or index links. When an entry is removed, its pool block is freed. When a new PUT succeeds, a pool block is allocated and the new entry points to it.

A hash index makes finding an entry by key practical for GET and for a PUT that replaces an existing key. A separate insertion-order list connects the active entries for FIFO and clock. These are different views of the same cached objects. The index answers *which entry has this key?* The policy list answers *which entry should be considered for removal?* The embedded block list answers *what region will become free if that entry is removed?*

The policy list is distinct from the embedded block list. Its order is the order of successful insertions, not the objects' addresses in the pool. An object inserted early can occupy a high-address block; a newer object can later use a low-address hole. Keeping these orders separate is what allows a victim selection to have a meaningful, sometimes surprising, effect on fragmentation.

Handling requests

GET

For a GET, the cache looks up the key in the index. If there is an active entry, it reports a hit and marks the object as referenced for clock. If there is no entry, it reports a miss. Neither result allocates space, frees a block, changes the insertion order, or moves the clock hand. A GET can affect a *future* eviction without changing the pool immediately.

PUT

A PUT has more work to do. It first checks whether the requested object could fit even if the pool were completely empty. The initial block header takes space, so the maximum object is smaller than the total pool size. If the request is too large, the cache rejects it immediately. Evicting other objects cannot make an impossible allocation possible.

If the key is already cached and the new size is possible, the PUT is an update. The old entry is removed and its pool block is freed before allocating the replacement. This removal is not an eviction chosen by a replacement policy. Rejecting an oversized update *before* removing the old entry is important: the original object remains in the cache when its replacement cannot fit.

The cache then asks the pool allocator for a sufficiently large block. If one is available, it may be split, and the cache records the new object under its key. If allocation fails, the cache selects a victim, removes that entry from the index and policy list, and frees its block. Freeing may coalesce neighboring blocks. The allocator tries again against the changed embedded list.

The sequence repeats until an allocation succeeds. One freed object's block may still be too small or in the wrong place; several victims may have to leave before their space forms a region large enough for the request. A non-oversized PUT can eventually succeed because freeing all cached objects would restore the initial free region. On success, the new object becomes the newest insertion and starts with its clock reference bit set.

```
PUT → check possible size → handle existing key → try to allocate
                                                        |
                                                if no block fits
                                                        ↓
                                          choose victim → free → merge if adjacent
                                                        |
                                                        └──────────→ try again
```

This retry is the central interaction in the project. Eviction policy chooses an object, which determines *where* the pool changes. Placement policy searches the resulting block list, which determines *where* the new object goes and what remains for later requests.

Placement and eviction policies

Choosing a free block

First-fit walks the embedded list from the beginning and selects the first free block large enough for the object. **Best-fit** examines the entire list and selects the smallest free block large enough. Neither looks ahead at future requests. Both use the same splitting and coalescing rules; they differ in the region chosen when more than one region fits.

Suppose the first usable hole has capacity 240 and a later one has capacity 120. A request for 100 fits in either. First-fit takes the 240-byte hole; best-fit takes the 120-byte hole, leaving the larger region available. That might help a later large request. On another workload, the smaller leftovers produced by best-fit might instead be hard to reuse. There is no policy that wins simply because it sounds more economical.

Choosing an object to remove

FIFO removes the oldest object among those still cached. A successful PUT adds a new entry at the end of the insertion order; an update replaces the old entry with a new insertion. GET hits do not change FIFO order. FIFO is straightforward, but it may evict an object that was just requested.

Clock uses the same set of active entries and a hand that remembers where the search left off. Successful PUTs and GET hits set an entry's reference bit. When clock needs a victim, it checks the entry at its hand. If the bit is set, clock clears it, advances, and gives the entry another chance. If the bit is clear, that entry is the victim. The hand persists across PUTs, so each sweep continues from the preceding state rather than starting again at the oldest entry.

A reference bit is evidence of recent use, not a prediction. If all bits are set, clock can clear them all before it finds a victim on another pass. It approximates the idea of keeping recently used objects without maintaining an exact access history. Which objects survive will depend on the sequence of GETs and PUTs, as well as on where the chosen victims' blocks lie in the pool.

A trace that changes the victim

The provided `clock_test.trace` makes the difference between FIFO and clock observable with a 1,024-byte pool and one client. It begins by storing four 200-byte objects, A through D. None of the remaining free blocks can hold another 200-byte object. Storing E therefore requires an eviction. In this run, both policies evict A and place E in its block.

Next comes `GET B`, which hits and sets B's reference bit. Then the trace stores F, forcing another eviction. FIFO selects B because B is now the oldest insertion, even though it was just requested. Clock finds B recently referenced, gives it a second chance, and evicts C instead. Both runs have made two evictions, but they end with different contents and different occupied pool regions. A subsequent GET for B would miss in the FIFO run and hit in the clock run.

```
PUT A 200       PUT B 200       PUT C 200       PUT D 200
PUT E 200       GET B           PUT F 200

FIFO:   evict A for E; later evict B for F
Clock:  evict A for E; later evict C for F
```

The example keeps the placement policy fixed. It isolates the effect of replacement: B and C occupy different pool blocks, so choosing one rather than the other changes the free-space layout too. A trace with free regions of different sizes can expose the other axis, first-fit versus best-fit. Comparing policies means following the history of blocks and entries, not just checking whether two final counters differ.

Running and interpreting the program

Build the program and start with the trace above:

```
make
./cachelab 1024 test_tr/clock_test.trace 1 -a first -e fifo
./cachelab 1024 test_tr/clock_test.trace 1 -a first -e clock
```

To see the difference in GET results, copy `test_tr/clock_test.trace` to a new trace and add `GET B` after its final `PUT F 200`. Run both policies again using the new trace path. The added GET reports MISS with FIFO and HIT with clock; the provided trace ends before that lookup.

The arguments specify total pool bytes, trace file, and number of client threads; `-a` selects `first` or `best`, and `-e` selects `fifo` or `clock`. The pool-size number includes block headers. Repeat other single-client traces with all four combinations to compare placement and eviction together.

The output records what happened as each request ran. `ALLOC` gives the pool offset and requested size of a new object. `EVICT` identifies the removed key and its former location. `COALESCE` reports a merge of adjacent free blocks. `HIT` and `MISS` report GET results. `UPDATE` identifies replacement of an existing key; `REJECT` identifies a PUT that could not fit even in an empty pool. The final `STATS` line counts hits, misses, evictions, and rejects.

`STATS` also reports final occupancy as the sum of the *requested payload sizes* still cached, over the total pool size. It does not include block headers or unused capacity absorbed inside allocated blocks. The event log, rather than that one number, shows the sequence of placements, removals, and merges that led to the final state.

For each run, follow a PUT that cannot immediately allocate: which victim was chosen, what block became free, whether it merged with a neighbor, and which free block the allocator then used. Look at later GETs to see the cost of that victim choice. Comparing these histories makes the policy results explainable.

Provided concurrency

The program accepts one to eight client threads, but thread creation, request dispatch, and synchronization are provided infrastructure. The worker claims a request and protects the cache while that entire GET or PUT executes. Its output lines stay together. You can reason about a request as one complete change to shared state rather than deciding where to place locks or writing a thread protocol. The central work remains the embedded block list and its interaction with placement and eviction.

With one client, requests execute in trace order. Start there: the resulting block layout and policy choices are repeatable. With several clients, workers claim requests in trace order but may enter the protected cache in a different order. A PUT by one client can therefore affect what another client's GET finds. The request number and client ID on each event identify what ran; different runs may produce different valid histories.

Concurrency matters because the pool, cache entries, and eviction order are shared, and each finished request must leave them consistent. The scaffold handles the mechanics that make each request atomic. Your job is to understand the state changes inside that protected request, first in a single-client trace and then in runs where multiple clients share the same cache.

Connection to the memory-management module

The earlier chapters on address spaces describe the memory a process sees. `cachelab` obtains a region of that memory and subdivides it for cached objects. The operating system supplies and maps memory for the process; managing the blocks *inside this pool* is the program's job. That distinction is why a header in the pool describes one of this cache's blocks, not an OS page or a physical frame.

OSTEP Chapter 17 studies how to manage free space when allocations have different sizes. The embedded block list in this project makes that problem concrete: you can see headers divide a pool into blocks, observe where a search policy places a request, and watch splitting and coalescing change the available regions. External fragmentation is not just a definition here. It is the reason a PUT may fail despite free capacity elsewhere in the pool.

The module also studies replacement decisions. FIFO and clock draw on the policies in OSTEP Chapter 22: when space is needed, a system chooses what to remove using the information it has about past use. In this cache, selecting a victim has two consequences at once. The object may be missed on a later GET, and its particular pool block becomes free for future allocations. The exercise connects replacement to the allocator's changing view of free space.

This is an analogy, not a virtual-memory simulator. Virtual-memory pages and physical frames have fixed sizes; the objects in this cache vary in size. A cachelab GET miss does not trigger disk I/O or bring in a page. The relevant operating-systems ideas are the mechanisms and tradeoffs you can inspect directly: managing a limited space, choosing a region for each allocation, recovering regions on free, and deciding what to retain when capacity is contested.

By the end, you should be able to explain a result in terms of the state that produced it. Why did this PUT need two evictions? Why did freeing one object merge blocks but freeing another did not? Why did FIFO and clock retain different keys, or first-fit and best-fit leave different holes? Those explanations are the point of running the same workload under several policies.

Project 2 materials

The **Project 2 Overview**, **Project 2 Prompt**, and **Pool Tutorial: From Linked Lists to Embedded Free Lists** are all available in **Modules in Canvas**.